

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_23

23. Mediator Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Mediator pattern.

GoF Definition

It defines an object that encapsulates how a set of objects interact. The Mediator pattern promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.

Concept

A mediator is an intermediary through whom different objects talk to each other. It takes responsibility for controlling and coordinating the interactions of a specific group of objects that cannot refer to each other explicitly. As a result, you can reduce the number of direct interconnections and promote loose coupling among them in your application.

Real-Life Example

When an airplane needs to take off, a series of verifications take place. These kinds of verifications confirm that all components and parts (which are dependent on each other) are in perfect condition.

You can also consider the case when the pilots of different airplanes (that are approaching or departing the terminal area) communicate with the airport towers. They do not explicitly communicate with other pilots in different airlines. They send their status to the controlling tower. These towers send signals to confirm who can take off (or land). These towers do not control the whole flight. They enforce constraints only in the terminal areas.

Computer World Example

When a client processes a business application, you may need to implement some constraints. For example, suppose you have a form where clients need to supply their user IDs and passwords to access the accounts. In the same form, you may include other mandatory fields such as e-mail ID, communication address, age, and so on. Let's assume you are applying the constraints as follows.

Initially, you check whether the user ID supplied by a user is a valid one. If it is a valid ID, then only the password field are enabled. After supplying these two fields, you may need to check whether the e-mail ID is pro-

vided by the user. Let's assume further that after providing all this information (a valid user ID, a password, a correctly formatted e-mail ID, and so on), the Submit button is enabled. In other words, the Submit button is enabled only if the user supplies a valid user ID, password, valid email ID, and other mandatory details. You can also ensure that the user ID is an integer, so if a user by mistake provides any character in that field, the Submit button will stay in disabled mode. The Mediator pattern becomes handy in such a scenario.

When a program consists of many classes and the logic is distributed among them, the code becomes harder to read and maintain. In these scenarios, if you want to bring changes into the system's behavior, it can be difficult unless you use the Mediator pattern. Here are a few more examples:

- The `execute()` method inside the `java.util.concurrent.Executor` interface follows this pattern.
- The `javax.swing.ButtonGroup` class is another example that supports this pattern. This class has a method named `setSelected()` which ensures that a user can provide a new selection.
- The different overloaded versions of various `schedule()` methods of the `java.util.Timer` class also can be considered as following this pattern.

Implementation

Wikipedia describes the Mediator pattern as shown in Figure 23-1 (which is basically adopted from the GoF).

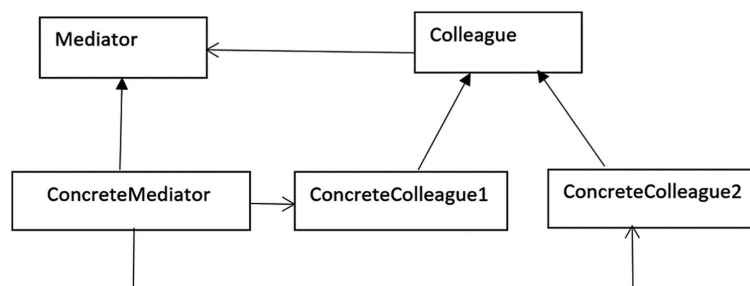


Figure 23-1 Mediator pattern example

The participants are described as follows:

- **Mediator:** It defines the interface that provides communication among `Colleague` objects.
- **ConcreteMediator:** It knows and maintains the list of `Colleague` objects. It implements the `Mediator` interface and coordinates the communication among the `Colleague` objects.
- **Colleague:** It defines the interface for communication with other colleagues.
- **ConcreteColleague(s):** A concrete colleague must implement the `Colleague` interface. These objects communicate with each other through the mediator.

In demonstration 1, I replaced `Colleague` with `Person` and `ConcreteColleague(s)` with `Employee` and `Outsider`. The reason is simple: I want to show how a mediator can control an outsider in an organization's chat server.

In this example, there are three different participants named `amit`, `sohel`, and `joseph` who can communicate with each other through a chat server. This chat server plays the role of a concrete mediator in this scenario.

In the upcoming example, `Mediator` is a Java interface. Refer to the comments for your easy understanding. It is defined as follows:

```
interface Mediator {
    // To register an employee
    void register(Person person);
    // To send a message from one employee to
    // another employee
    void connectEmployees(Person fromPerson,
                          Person toPerson,
                          String msg)
                          throws InterruptedException;
    // To display currently registered members.
    void displayDetail();
}
```

The `ChatServer` class implements this interface. It maintains the list of registered participants. So, inside this class, you see the following line of code:

```
List<Person> participants = new ArrayList<Person>();
```

Apart from this, the mediator allows only registered users to communicate with each other and post messages successfully. So, the `connectEmployees(...)` method in `ConcreteMediator` checks whether both the sender and receiver are registered users or not. The method is defined as follows:

```
@Override
public void connectEmployees(Person fromPerson, Person toPerson, String msg) throws Int
// If the sender is a registered user
if (participants.contains(fromPerson)) {
// If the receiver is a registered user
if (participants.contains(toPerson)) {
    System.out.println(fromPerson.getName() + " posts: " +
        Msg + " at:" + LocalDateTime.now());
    Thread.sleep(1000);
    // Target receiver receives this message.
    toPerson.receiveMessage(fromPerson, msg);
} else {
    System.out.print(fromPerson.getName() + ", you cannot
        send message to " + toPerson.getName() + ".");
    System.out.println("He/she is NOT a registered user.");
}
}
// The message sender is not a registered user.
else {
    System.out.print("\n***ALERT: Everyone.***");
    System.out.println("An " + fromPerson.personType() + ": "
        + fromPerson.getName()+ " is trying to send
        some messages.");
}
}
```

In this example, there is another inheritance hierarchy, in which you use `Person` as an abstract class so that you cannot directly instantiate it. Instead, you can instantiate

objects from the concrete classes `Employee` or `Outsider`, which inherit from `Person`. This inheritance hierarchy is as follows:

```
// The abstract class-Person
abstract class Person {
    protected Mediator mediator;
    protected String name;
    public String getName() {
        return name;
    }
    // Constructor
    public Person(Mediator mediator) {
        this.mediator = mediator;
    }
    public void sendMessage(Person toEmp, String msg)
        throws InterruptedException {
        mediator.connectEmployees(this, toEmp, msg);
    }
    public abstract String personType();
    public void receiveMessage(Person fromEmp, String msg) {
        System.out.println(this.name + " has read " +
            fromEmp.getName() + "'s message.");
    }
}

// Employee
class Employee extends Person {
    public Employee(Mediator mediator, String name) {
        super(mediator);
        this.name = name;
    }
    @Override
    public String personType() {
        return "Employee";
    }
}

// Unknown participant-an outsider.
class Outsider extends Person {
    // Constructor
    public Outsider(Mediator mediator, String name) {
        super(mediator);
        this.name = name;
    }
    @Override
    public String personType() {
        return "Outsider";
    }
}
```

Note Following the core architecture of a basic Mediator pattern, I use two different concrete classes to demonstrate the following fact: you should NOT assume that the communicating objects should appear from the same class only.

Inside the client code, there are different participants. Here are some examples:

```
Person amit = new Employee(mediator, "Amit");
Person sohel = new Employee(mediator, "Sohel");
Person joseph = new Outsider(mediator, "Joseph");
```

These people can communicate among themselves through a chat server. So, before passing the messages, they first register themselves to the chat server as follows:

```
// Registering participants  
mediator.register(amt);  
mediator.register(sohel);  
mediator.register(joseph);
```

At the end of the program, you introduced two more people named `todd` and `jack`; `todd` is an `Employee` class object and `jack` is an `Outsider` class object. But neither of them registered themselves with the mediator object, so the mediator does not allow them to post any message to the target recipient.

Obviously, the `Outsider` instance of `jack` will be able to send the message properly if he registers himself to the mediator before sending the message as follows:

```
mediator.register(jack); // Not shown in Demonstration1  
jack.sendMessage(joseph, "Hello Joseph...");
```

And the same comment applies to `todd`.

Class Diagram

Figure 23-2 shows the important parts of the class diagram.

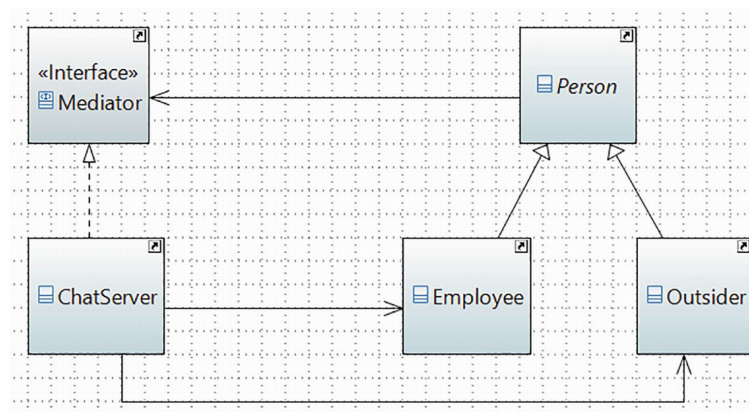


Figure 23-2 Class diagram

Package Explorer View

Figure 23-3 shows the high-level structure of the program.

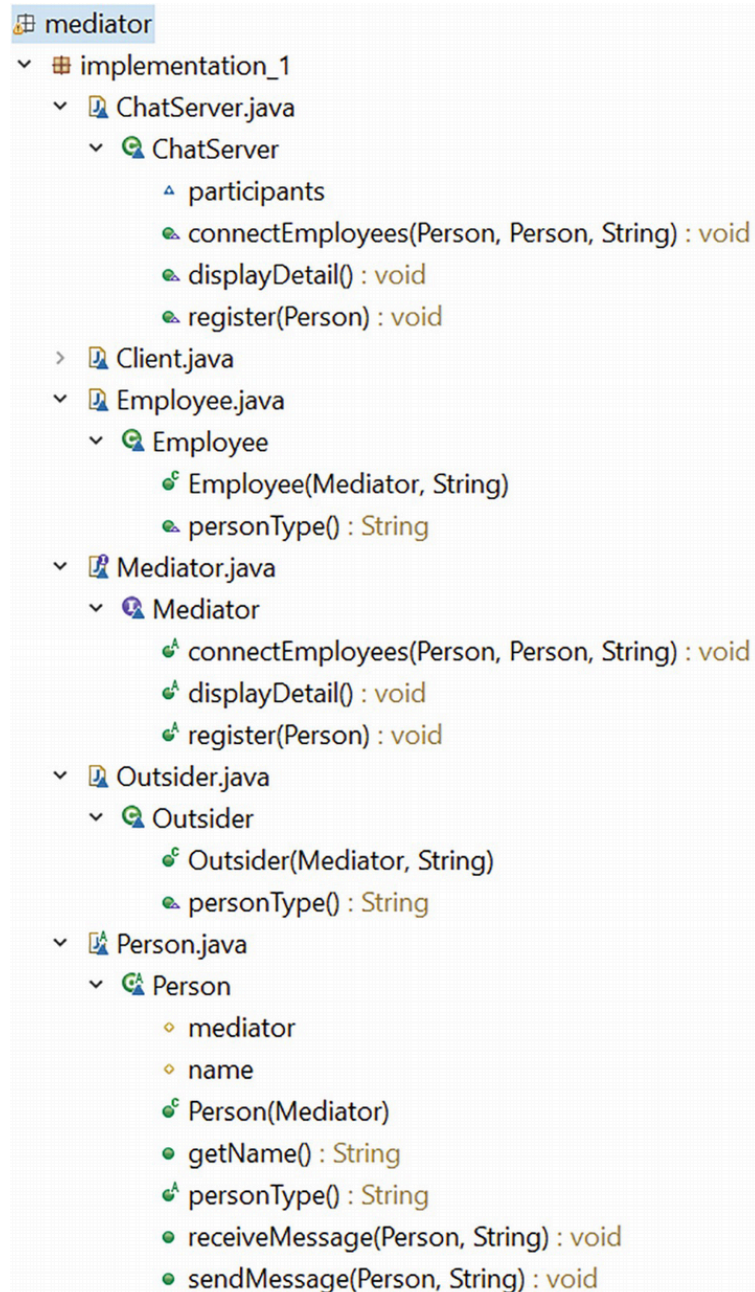


Figure 23-3 Package Explorer view

Demonstration 1

Here's the complete demonstration. All parts are stored inside the package `jdp3e.mediator.implementation_1`.

```
// Mediator.java
interface Mediator {
    // To register an employee
    void register(Person person);
    // To send a message from one employee to
    // another employee
    void connectEmployees(Person fromPerson, Person toPerson,
        String msg) throws InterruptedException;
    // To display currently registered members.
    void displayDetail();
}

// ChatServer.java
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
// ConcreteMediator
```

```

class ChatServer implements Mediator {
    List<Person> participants = new ArrayList<Person>();
    @Override
    public void register(Person employee) {
        participants.add(employee);
    }
    @Override
    public void displayDetail() {
        System.out.println("At present, the registered employees are:");
        for (Person person : participants) {
            System.out.println(person.getName());
        }
    }
    @Override
    public void connectEmployees(Person fromPerson,
                                Person toPerson,
                                String msg)
        throws InterruptedException {
        // If the sender is a registered user
        if (participants.contains(fromPerson)) {
            // If the receiver is a registered user
            if (participants.contains(toPerson)) {
                System.out.println(fromPerson.getName()
                                    + " posts: " + msg + " at:" +
                                    LocalDateTime.now());
                Thread.sleep(1000);
                // Target receiver receives
                // this message.
                toPerson.receiveMessage(fromPerson, msg);
            } else {
                System.out.print(fromPerson.getName() +
                                   ", you cannot send the message to " +
                                   toPerson.getName() + ".");
                System.out.println("He/she is NOT a
                                   registered user.");
            }
        }
        // The message sender is not a registered user.
        else {
            System.out.print("\n****ALERT: Everyone****");
            System.out.println("An " +
                                fromPerson.personType() + ": " +
                                fromPerson.getName() + " is trying to
                                send some messages.");
        }
    }
}

// Person.java
// The abstract class-Person

abstract class Person {
    protected Mediator mediator;
    protected String name;
    public String getName() {
        return name;
    }
    // Constructor
    public Person(Mediator mediator) {
        this.mediator = mediator;
    }
    public void sendMessage(Person toEmp, String msg) throws
        InterruptedException {

```

```

        mediator.connectEmployees(this, toEmp, msg);
    }

    public abstract String personType();
    public void receiveMessage(Person fromEmp, String msg) {
        System.out.println(this.name + " has read " +
            fromEmp.getName() + "'s message.");
    }
}

// Employee.java
class Employee extends Person {
    public Employee(Mediator mediator, String name) {
        super(mediator);
        this.name = name;
    }
    @Override

    public String personType() {
        return "Employee";
    }
}

// Outsider.java
// Unknown participant-an outsider.
class Outsider extends Person {
    // Constructor
    public Outsider(Mediator mediator, String name) {
        super(mediator);
        this.name = name;
    }
    @Override
    public String personType() {
        return "Outsider";
    }
}

// Client.java
class Client {
    public static void main(String[] args) throws
        InterruptedException {
        System.out.println("***Mediator Pattern Demonstration-1.***\n");
        ChatServer mediator = new ChatServer();
        Person amit = new Employee(mediator, "Amit");
        Person sohel = new Employee(mediator, "Sohel");
        Person joseph = new Outsider(mediator, "Joseph");
        // Registering participants
        mediator.register(amit);
        mediator.register(sohel);
        mediator.register(joseph);
        // Displaying the participant's list
        mediator.displayDetail();
        System.out.println("Communication starts among participants...");
        amit.sendMessage(sohel, "Hi Sohel, can we discuss the mediator
            pattern?");
        amit.sendMessage(joseph, "Hi Joseph, how do you do?");
        sohel.sendMessage(amit, "Hi Amit. Yup, we can discuss now.");
        joseph.sendMessage(amit, "Hello, friend.");
        // Another employee-Todd. He does not register
        // to the mediator.
        Person todd = new Employee(mediator, "Todd");
        // Todd is an employee, but not a registered user.
        // So, he cannot send a message to a registered
        // user. For the same reason, he cannot receive any
        // message from a registered user.
        todd.sendMessage(joseph, "Hello Joseph...");
    }
}

```



```
        amit.sendMessage(todd, "Hello Todd...");
        // An outsider person tries to participate
        Person jack = new Outsider(mediator, "Jack");
        // This message also cannot reach Joseph,
        // because Jack is not the registered user.
        jack.sendMessage(joseph, "Hello Joseph...");
    }
}
```

Output

Here's the output:

```
***Mediator Pattern Demonstration-1.***
At present, the registered employees are:
Amit
Sohel
Joseph
Communication starts among participants...
Amit posts: Hi Sohел, can we discuss the mediator pattern? at:2021-10-05T13:11:43.86026
Sohel has read Amit's message.
Amit posts: Hi Joseph, how do you do? at:2021-10-05T13:11:44.870799500
Joseph has read Amit's message.
Sohel posts: Hi Amit. Yup, we can discuss now. at:2021-10-05T13:11:45.880435500
Amit has read Sohел's message.
Joseph posts: Hello, friend. at:2021-10-05T13:11:46.891890900
Amit has read Joseph's message.
***ALERT: Everyone.***An Employee: Todd is trying to send some messages.
Amit, you cannot send the message to Todd.He/she is NOT a registered user.
***ALERT: Everyone.***An Outsider: Jack is trying to send some messages.
```

Analysis

Note that only registered users can communicate with each other and post messages successfully. The mediator does not allow others into the system. (Notice the last few lines of the output).

Point To Remember

You should not assume that there should be always one-to-one communication. This is because the GoF book says that a mediator replaces many-to-many interaction with one-to-many interaction. But in this chapter, I assume that these are private messages and thus should not be broadcasted to everyone, so I show you an example where the mediator sends the messages to an intended receiver only. The mediator broadcasts the messages to warn others only when an outsider (or an unregistered person) tries to post messages on this chat server.

Additional Implementation

Here's another implementation in which I add the following constraint: once a participant reads a message, the text color changes. In the second edition of the book, I showed you a modified application that could check the online status of the target receiver before a sender sends a message to a receiver. I changed the constraint because of the following reasons:

- You already get the idea that you can add the additional validation inside the mediator (more specifically inside the `connectEmployees(...)` method). So, I leave this exercise to you.
- This time I want to mimic another real-world scenario: WhatsApp users know that once you read a message, the text color changes. I want to demonstrate a similar application.
- I want to show an alternative design. In demonstration 1, I create and register users/participants inside the client code. In the upcoming example, I change the approach. Now a participant self-registers back to the mediator inside the constructor (see the code in bold):

```
// Constructor
public Employee(ChatServer chatServer, String name) {
    this.chatServer = chatServer;
    this.name = name;
    chatServer.registerEmployee(this);
}
```

To make this implementation short and simple, this time I do not use an interface/abstract class. The four participants, `ChatServer`, `Employee`, `Text`, and `Client`, are concrete classes. You can easily guess that `ChatServer` plays the role of the mediator. Since the overall intent is similar to demonstration 1, you can directly jump into the code.

Demonstration 2

All classes are inside the package `jdp3e.mediator.implementation_2`. Here is the implementation:

```
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
// Mediator
// ChatServer.java
class ChatServer {
    Text text;
    List<Employee> employees;
    public ChatServer() {
        employees = new ArrayList<Employee>();
    }
    public void registerEmployee(Employee employee) {
        employees.add(employee);
    }
    public void registerText(Text text) {
        this.text = text;
    }
    public void displayEmployees() {
        System.out.println("At present, the registered employees are:");
        for (Employee emp : employees) {
            System.out.println(emp.getName());
        }
    }
    public void postMessage(Employee fromEmp, Employee toEmp,
        String msg) throws InterruptedException {
        // Include some validation logic-if necessary
        // Posting the message
        System.out.println(fromEmp.getName() + " posts: " +
            msg + " at:" + LocalDateTime.now());
        text.makeDefaultColor();
        toEmp.receiveMessage(fromEmp, msg);
    }
}
```

```

        // Some logic (skipped) to ensure that
        // the message is received and read.
        text.makeBlueText();
    }
}
// Employee.java
class Employee {
    private ChatServer chatServer;
    protected String name;
    public String getName() {
        return name;
    }
    // Constructor

    public Employee(ChatServer chatServer, String name) {
        this.chatServer = chatServer;
        this.name = name;
        chatServer.registerEmployee(this);
    }
    public void sendMessage(Employee toEmp, String msg)
    throws InterruptedException {
        chatServer.postMessage(this, toEmp, msg);
    }
    public void receiveMessage(Employee fromEmp, String msg){
        System.out.println(this.name + " has read " +
            fromEmp.getName() + "'s message now.");
    }
}
// Text.java
class Text {
    private ChatServer chatServer;
    String color;
    public Text(ChatServer chatServer, String color) {
        this.chatServer = chatServer;
        this.color=color;
        // self-registering back to the
        // mediator(chatServer)
        chatServer.registerText(this);
    }
    public void makeDefaultColor() {
        System.out.println("Message text has a default
            color: "+ color);
    }
    public void makeBlueText() {
        System.out.println("***Message text becomes blue.***\n");
    }
}
class Client {
    public static void main(String[] args) throws
        InterruptedException {
        System.out.println("***Mediator Pattern Demonstration-2.***\n");
        ChatServer mediator = new ChatServer();
        Employee amit = new Employee(mediator, "Amit");
        Employee sohel = new Employee(mediator, "Sohel");
        Employee kate = new Employee(mediator, "Kate");
        Text text = new Text(mediator, "black");
        // Displaying the participant's list
        mediator.displayEmployees();
        System.out.println("\nCommunication starts among participants...");
        amit.sendMessage(sohel, "Hi Sohel, can we discuss the mediator
            pattern?");
        sohel.sendMessage(amit, "Hi Amit. Yup, we can discuss now.");
    }
}

```

```

        amit.sendMessage(kate, "Hi Kate!");
    }
}

```

Output

Here's the new output:

```

***Mediator Pattern Demonstration-2.***
At present, the registered employees are:
Amit
Sohel
Kate
Communication starts among participants...
Amit posts: Hi Sohел, can we discuss the mediator pattern? at:2021-10-04T14:44:16.03615
Message text has a default color: black
Sohel has read Amit's message now.
***Message text becomes blue.***
Sohel posts: Hi Amit. Yup, we can discuss now. at:2021-10-04T14:44:16.037156100
Message text has a default color: black
Amit has read Sohел's message now.
***Message text becomes blue.***
Amit posts: Hi Kate! at:2021-10-04T14:44:16.037156100
Message text has a default color: black
Kate has read Amit's message now.
***Message text becomes blue.***

```

NoteA few lines of the previous output are bold. They are used to demonstrate the impact of the modified program (demonstration 2).

Analysis

You can see that once a participant reads the message, the text color changes to blue. Applications such as WhatsApp follow a similar principle.

Q&A Session

23.1 Why are you complicating things? In demonstration 1, each of the participants could talk to each other directly, and you could bypass the use of a mediator. Is this correct?

In this example, you have only three *registered* participants and the mediator allows them only to communicate with each other. So, it may appear that since the participants are only three, they could communicate with each other directly. But you know that an organization can have many more employees in general.

In demonstration 1, you see that before sending a message to a target receiver, the mediator checks whether they are a registered user or not. In addition, it also checks whether the sender belongs to the same organization or is an outsider. So, you understand that a mediator can perform this kind of validation. Let's assume that you want an additional constraint:

- A participant can send a message to a target participant if and only if the target participant stays in online mode only (which is the com-

mon scenario for a chat server).

If you do not use a Mediator pattern, it is not sufficient to check whether the participant is a valid user or not; in addition to this, you need to check the target recipient's online status before you post a message. And if the number of participants keeps growing, can you imagine the complexity of the system? A mediator can rescue you from such a scenario because you can put the validation criteria inside the mediator. Figures 23-4 and 23-5 depict the scenario better.

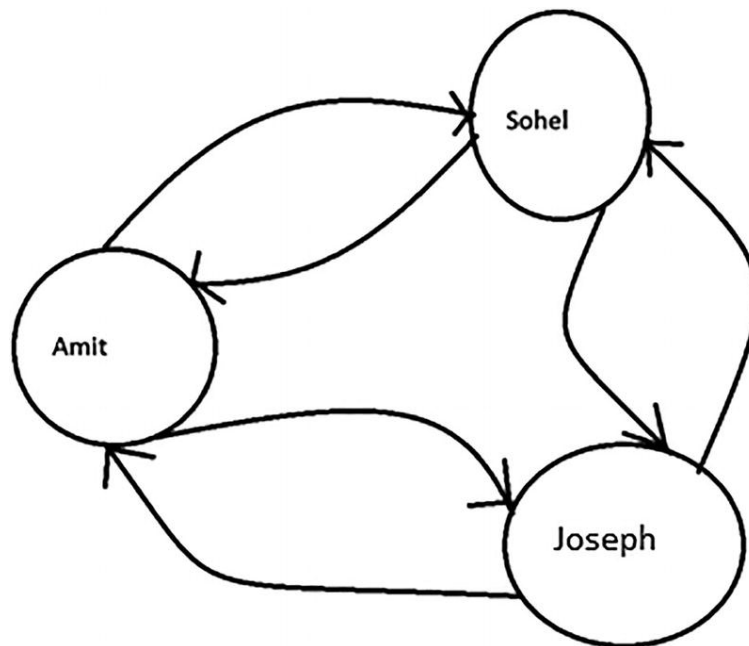


Figure 23-4 Case 1: Without using a mediator

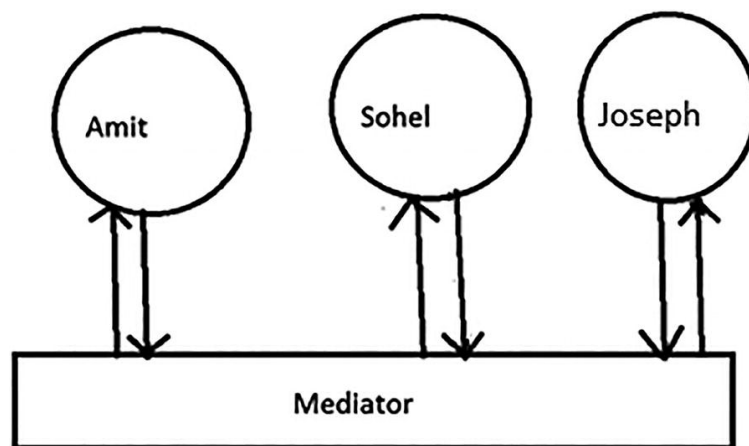


Figure 23-5 Case 2: With a mediator

23.2 What are the advantages of using the Mediator pattern?

Here are some advantages:

- You can reduce the complexity of objects communicating in a system.
- The pattern promotes loose coupling so objects can be reused.
- The pattern reduces the number of subclasses in the system.
- You replace a many-to-many relationship with a one-to-many relationship, so the code is much easier to read and understand. And as an obvious effect of this, maintenance becomes easier.
- You can provide a centralized control with this pattern.
- In short, it is always a good aim to remove the tight coupling from your code, and the Mediator pattern scores high in this context.

23.3 What are the disadvantages of using the Mediator pattern?

The following points talk about the challenges:

- In some cases, implementing the proper encapsulation becomes tricky, and the mediator object's architecture becomes complex.
- Sometimes maintaining a complex mediator becomes a big concern.

23.4 If you need to add a new rule or logic, you can directly add it to the mediator. Is this correct?

Yes.

23.5 I am finding some similarities between the Facade pattern and the Mediator pattern. Is this correct?

Yes. I know that demonstration 2 gives you the idea. Steve Holzner in his book *Design Pattern for Dummies* also mentions the similarity by describing the Mediator pattern as a multiplexed Facade pattern. In the case of the Mediator pattern, instead of working with an interface of a single object, you make a multiplexed interface among multiple objects to do smooth transitions.

23.6 In this pattern, you are reducing the number of interconnections among various objects. What are the key benefits you have achieved because of this reduction?

More interconnections among objects can create a monolithic system that becomes difficult to change (because the behaviors are distributed among many objects). As a side effect, you may need to create many subclasses to bring those changes into the system.

23.7 In demonstration 1, you used `Thread.sleep(1000)`. What is the reason behind this code?

Good catch. It is not necessary so you can ignore it. I use it to mimic a real-life scenario. I assume that participants are posting the messages after reading them properly, and this activity takes a minimum of 1 second.

23.8 Can I conclude the following: if a class calls methods from multiple objects, it is a mediator?

Not at all. You must note the key purpose of a mediator is to simplify the complex communications among objects in a system. Always keep in mind the GoF definition and the corresponding concepts.

23.9 In demonstration 1, the send methods for mediator and persons (i.e., employees/outside) are different. One is `connectEmployees(...)` and the other is `sendMessage(...)`. Do I need to follow any specific naming convention?

Both methods can have the same name, but I want you to note that it is not mandatory.

23.10 I understand that both the Mediator pattern and the Observer pattern promote one-to-many communications. How should I distinguish between these two?

Remember that the mediator encapsulates the complex communication between objects. So, once it sends a notification to the registered objects, those objects also talk about their state changes to the mediator and the mediator can act accordingly. But in the case of the Observer pattern, once a subject sends a notification to multiple objects, it does not care about their state changes.

Previous chapter

< [22. State Pattern](#)

Next chapter

[24. Visitor Pattern](#) >